

CS4225/5425 Big Data Systems, Data Science

AY24/25 Sem 2. github.com/gerteck

(Big) Data science. Interdisciplinary field about processes and systems to extract knowledge or insights from data in various forms.

- **Processing** diff. types of data (large data volume, graph data, stream data).
- **Models of Computation:** MapReduce/Spark, Large graph processing engines, Streams and online algorithms.

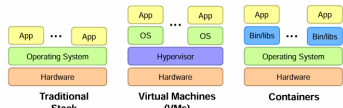
1. Introduction

- **Big Data (4 ‘V’s):** *Volume:* (Scale of data). *Velocity:* (Speed of new (streaming) data). *Variety:* (Different types). *Veracity:* (Accuracy, trustworthiness of data).
- **Challenges of 4 ‘V’s:** Performance, Cost, Reliability, Algorithm design complexity. Data integration challenges, multi-modal learning (Variety). Dirty and noisy data, data provenance, uncertainty (Veracity).
- Big data - Virtuous Product Cycle, where useful service → data to analyze → insightful action → useful service.

Utility Computing (Cloud Computing Infrastructure)

Cloud computing - scalable, elastic infrastructure for big data systems.

- **Utility Computing:** Computing resources as a metered service (“pay as you go”). Ability to dynamically provision virtual machines.
- **Scalable:** (“Infinite” capacity).
- **Elastic:** (Scale up or down on demand).



- **How: Virtual Machines (VMs):** Enable sharing of hardware resources, run each app in isolated virtual machine. High overhead as each VM has its own OS.
- **Containers:** Lightweight sharing of resources. Apps run in isolation but share same OS. Container encapsulates an app and environment.
- **Everything as a Service:**
 - *Infrastructure (IaaS):* User rents a virtual machine. (e.g. Amazon EC2).
 - *Platform (PaaS):* Provide hosting web apps. Cover hardware maintain, upgrade.
 - *Software (SaaS):* Pre-built apps. (e.g., Gmail). Covers entire stack.

Data science, cross-disciplinary, emerging research area w. applications in engineering, commerce, etc. Clouds as natural infrastructures for data science.

2. Principles of Big Data Systems

- **Data Center Infrastructure**
 - **Single Server:** A commodity hardware unit.
 - **Rack:** Dozens of servers, connect to rack switch (top-of-rack switch).
 - **Data Center:** Hundreds of racks, connect to data center switch (core switch).
- **Bandwidth vs. Latency**
 - **Bandwidth:** Max amt of data transmittable per unit time (e.g., in GB/s). Calculate as minimum bandwidth among all components in the data access path.
 - **Latency:** Time taken one packet travel src to dest (one-way) / round-trip (in ms). Sum of the latency caused by all components in the data access path.
 - **Throughput:** Rate data is actually transmitted in network in given time period.
- When transmitting (*large/small*) amount of data, (*bandwidth/latency*) - roughly how long transmission will take (bottleneck variable).
- **Storage Hierarchy in Data Centers:** based on capacity, latency, bandwidth.
 - **DRAM:** Fast but limited capacity. **Flash:** Intermediate btw. DRAM and disk.
 - **Disk:** Slow but large capacity. Disk reads more expensive than DRAM (higher latency, lower bandwidth).

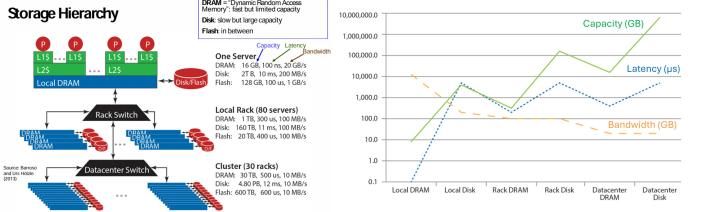
Data Flow Paths

Data flow paths depend on the location of the source and destination. Data access may cross several components, e.g. hard disk, DRAM, rack switch, cluster switch.

- E.g. Local DRAM → Local Disk in same server.
- Server A DRAM to server B DRAM in same rack (i.e. Local DRAM → Rack Switch → Server B’s DRAM)
- Transfer across different rack, same cluster etc. (i.e. Local DRAM → Rack Switch → Datacenter Switch → Server C’s Rack Switch → Server C’s DRAM).
- To load from disk, usually need to load into DRAM first. (i.e. Local Disk → Local DRAM → Rack Switch etc.)

Calculate latency, bandwidth based on data flow path. Approximate order of magnitude, for simplify, assume one-way communication, no failures.

Key metrics: different levels of storage and memory systems



Capacity increases, **Costs** (Latency increase, bandwidth decrease) increases from local to rack to data center, along storage hierarchy.

Building Efficient Big Data Systems

1. **Scale “Out”, Not “Up”:** Horizontal Scaling over vertical scaling. Combine many cheaper machines (out) over increase power of individual machine (up).
2. **Seamless Scalability:** Ideal (linear) scalability, performance increases proportionally to added resources.
3. **Move Processing to the Data:** As clusters limited bandwidth; move tasks to machine where data is stored.
4. **Process Data Sequentially, Avoid Random Access:** Disk seeks expensive; sequential access provides better throughput.

Challenges in Big Data Systems

1. **Machine Failures:** With enough scale, expect daily server machine failure.
2. **Synchronization:** Difficult due to unknown order of worker execution, interruptions, and shared data access. Tools like semaphores, conditional variables, and barriers are used to address synchronization issues.
3. **Programming Difficulty:** Concurrency hard to reason about, especially at scale. Debugging and managing failures add complexity.

API for Data Centers: Given challenges, “API” of data center abstracts system-level details from developers. Devs specifies computation needed performed, execution framework (“runtime”) handles actual execution. (Race condition, fault tolerance etc. handled by runtime.)

3. MapReduce

- The “API” of the data center abstracts system-level details from developers.
- Key operations:
 - **Map:** Perform an operation on each item.
 - **Reduce:** Aggregate intermediate results.
 - Execution frameworks (e.g., Hadoop, Spark) handle task assignment, replication, synchronization, etc.